

DERİN ÖĞRENME

CNN's (Convolutional Neural Networks)

Evrişimli Sinir Ağları

Sınıflandırma ve Bağlanım Sözlüğü

- Örnek veya Girdi: Modele giren veri
- Tahmin veya Çıktı: Modelden size geri gelen herşey
- Hedef: Elinizdeki verilere göre modelinizin ideal durumda tahmin etmesini beklediğiniz şey
- Tahmin Hatası veya Kayıp Değeri: Modelin tahminleri ile hedef arasındaki mesafe.
- Sınıflar: Sınıflandırma probleminde olası etiket kümesidir.
- Etiket: Sınıflandırma probleminde bir örneğin sınıf bilgisidir.

Sınıflandırma ve Bağlanım Sözlüğü

- Gerçek Etiket: Veri setindeki her girdi için insanlar tarafından belirlenen hedef.
- İkili Sınıflandırma: Her örneğin iki sınıftan birine ait olduğu sınıflandırma problemidir.
- Çoklu Sınıflandırma: Her örneğin birden fazla sınıfa ait etiket değerine sahip olabildiği sınıflandırma problemidir.

Convolutional Neural Networks (CNNs)- Evrişimli Sinir Ağları

CNN veya ConvNET

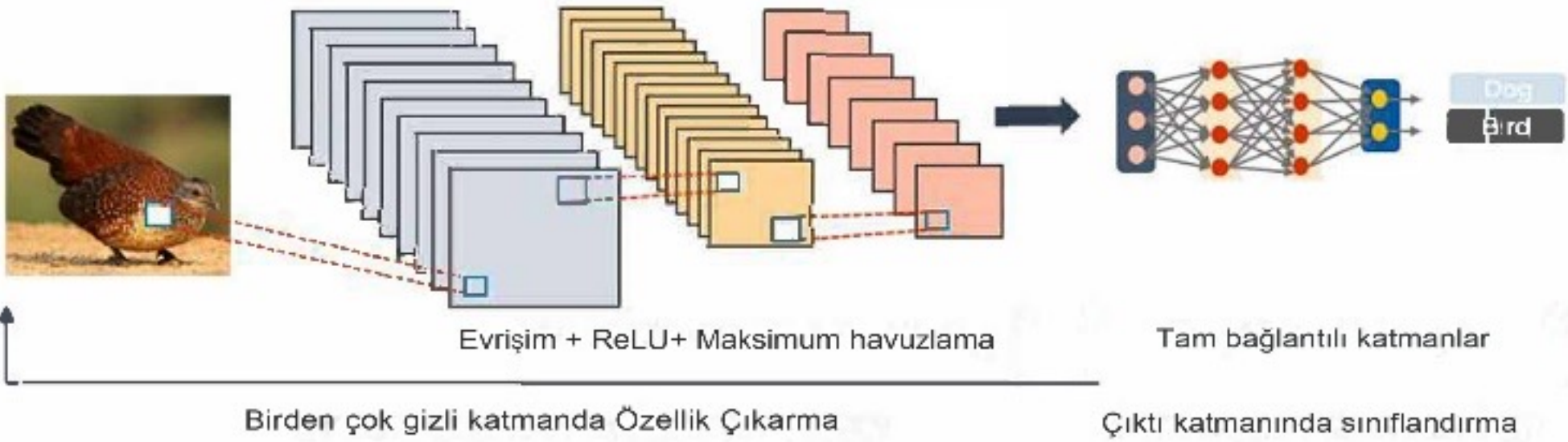
- CNN mimarisi canlıların görsel algısından esinlenilmiştir. CNN 2012 yılında popülerleşmeye başlasa da ön çalışmaları 1980'lere dayanmaktadır.
- Evrimsel sinir ağının temeli 1959'da Hubel ve Wiesel'in keşfi ile başlamıştır. Hubel ve Wiesel, hayvan görsel korteksinin hücreleri küç
- 1980 yılında Kunihiro Fukushima bu çalışmadan esinlenerek neokognitronu önermiştir. Bu ağ, CNN için ilk teorik model olarak kabul edilmektedir.

- 1990 yılında LeCun ve arkadaşları, el yazısı rakamları tanımak için LeNet-5 adı verilen modern CNN çerçevesini geliştirmiştir. Geriye yayılım algoritması ile eğitim, LeNet-5'in görsel desenleri tanımasına yardımcı olmuştur.
- Ancak o günlerde, donanım yetersizliğinden, CNN'nin karmaşık problemlerdeki performansı, sınırlı eğitim verileri, algoritmadaki yenilik eksikliği ve yetersiz hesaplama gücü nedeniyle eksikti.

- Son zamanlarda büyük etiketli veri kümelerimiz, yenilikçi algoritmalarımız ve güçlü GPU makinelerimiz var. 2012'de, bu yükseltmelerle, Krizhevsky ve diğerleri tarafından tasarlanan AlexNet adlı büyük derin CNN yapısı büyük başarılar göstermiştir.
- AlexNet'in başarısı, farklı CNN modellerinin icat edilmesinin yanı sıra bu modellerin bilgisayarlı görme ve doğal dil işleme gibi farklı alanlarda uygulanmasının yolunu açmıştır.

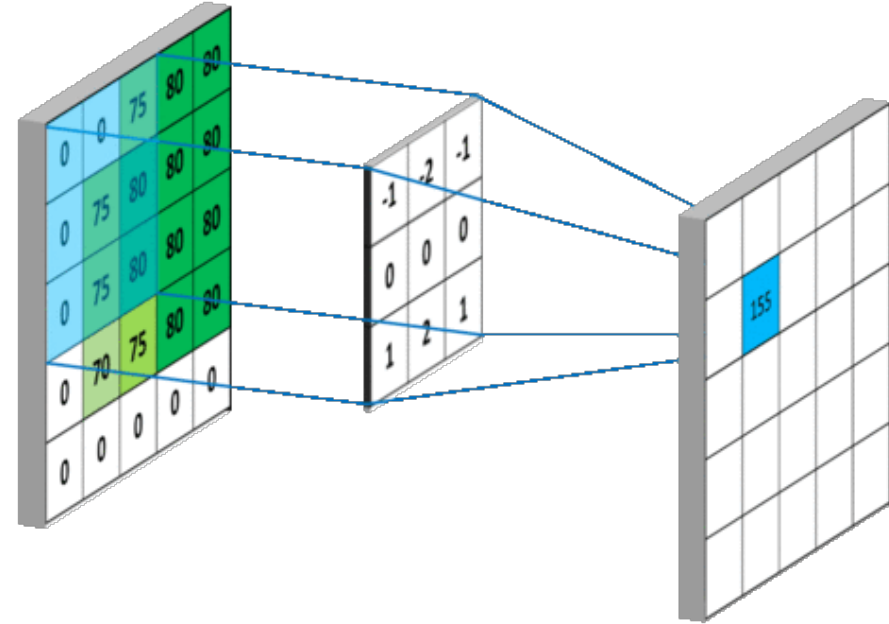
Evriřimli Sinir Ađının 3 Kavramı

- Geleneksel bir evriřimsel sinir ađı, bir veya birden fazla evriřim ve havuzlama katmanı blođunun ardından bir veya birden fazla tam bađlı (FC) katman ve bir ıkıř katmanından oluřur.
- Derin ileri beslemeli mimariye sahip ve inanılmaz genelleme yeteneđine sahip bir tr Yapay Sinir Ađı'dır (YSA).
- FC katmanlarına sahip diđer ađlarla karřılařtırıldığında, nesnelerin olduka soyutlanmış zelliklerini ğrenebilip zellikle mekânsal verileri tespit edip daha etkin bir řekilde tanımlayabilmektedir.



Evriřim Katmanı (Convolutional Layer)

- Herhangi bir CNN mimarisinin en önemli bileřenidir. Giriř görüntüsüyle (N boyutlu metrikler) evriřime uğrayarak bir çıkıř özellik haritası oluřturan bir dizi evriřimsel çekirdek (filtreler olarak da adlandırılır) ięerir.



Evrişim Katmanı (Convolutional Layer)

- Tam evrişim işleminden sonra bir özellik haritası oluşur.
- Diğer klasik ağlarda giriş vektör formatındadır fakat CNN'de giriş çok kanallı bir görüntüdür.
- Özellik haritasına aktivasyon fonksiyonu uygulanır.(ReLU)

Step-1

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1



0	1
-1	2



1		

Step-2

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1



0	1
-1	2



1	0	

Step-3

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1



0	1
-1	2



1	0	4

Step-4

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1



0	1
-1	2



1	0	4
4		

Step-5

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1



0	1
-1	2



1	0	4
4	1	

$$Y[i, j] = \sum_{m=1}^k \sum_{n=1}^k X[i + m - 1, j + n - 1] \cdot W[m, n]$$

1	0	4
4	1	1
1	1	2

$$h' = \left\lfloor \frac{h - f + p}{s} + 1 \right\rfloor$$

$$w' = \left\lfloor \frac{w - f + p}{s} + 1 \right\rfloor$$

Evrişim İşlemi

```
def convolution2d(input_matrix, kernel, stride=1):
    input_size = input_matrix.shape[0] # Giriş matrisi
    kernel_size = kernel.shape[0]      # Filtre boyutu

    # Çıktı Matrisi Boyutunu belirleme
    output_size = (input_size - kernel_size) // stride + 1
    output_matrix = np.zeros((output_size, output_size))

    # Evrişim İşlemi
    for i in range(0, output_size):
        for j in range(0, output_size):
            # Filtre ve Girdi değerlerii Çarpımı
            region = input_matrix[i:i+kernel_size, j:j+kernel_size]
            output_matrix[i, j] = np.sum(region * kernel)

    return output_matrix

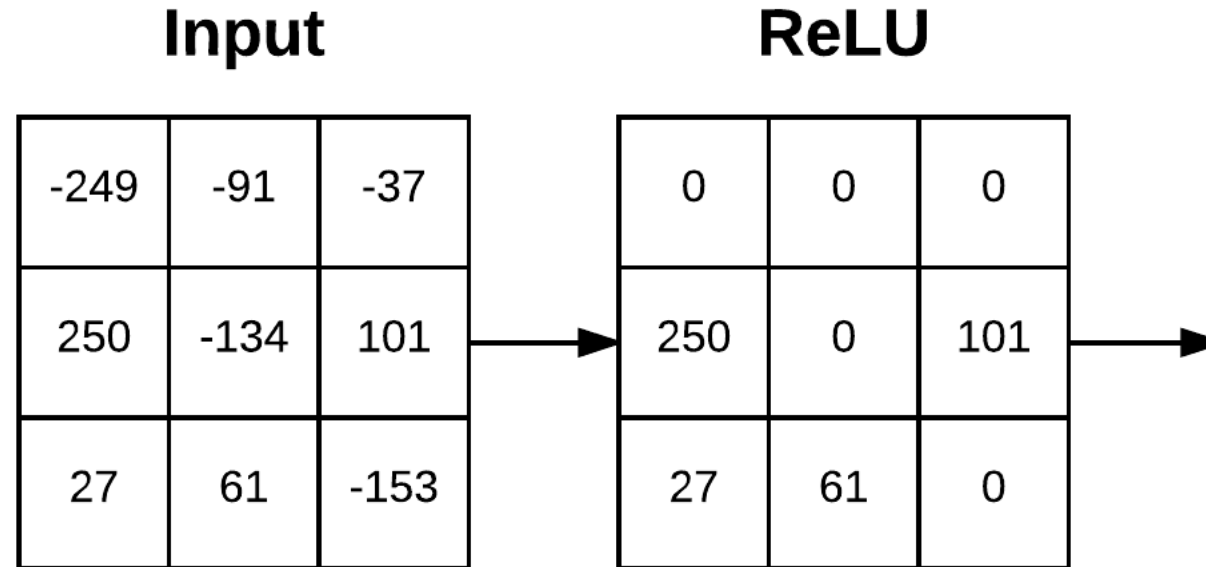
output_matrix = convolution2d(input_matrix, kernel)
```

```
# Sequential Model ile
model = Sequential()

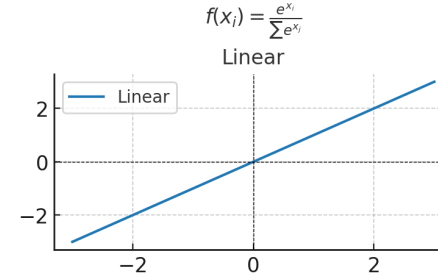
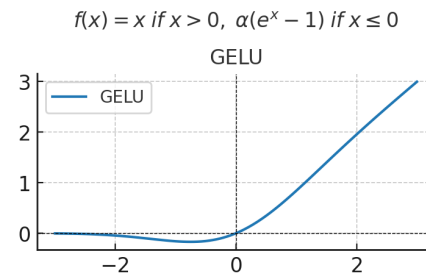
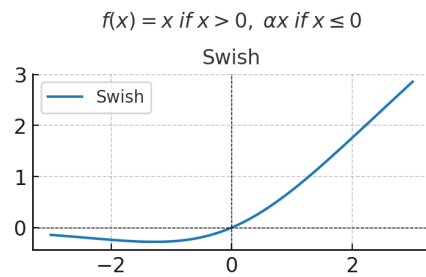
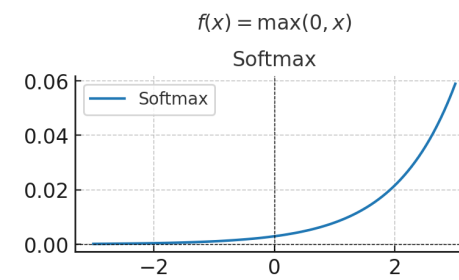
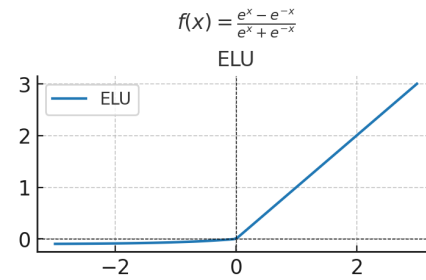
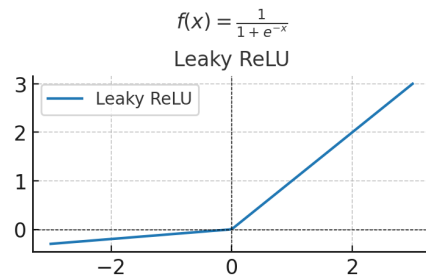
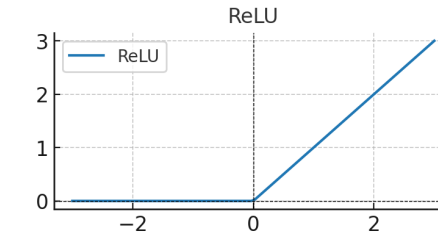
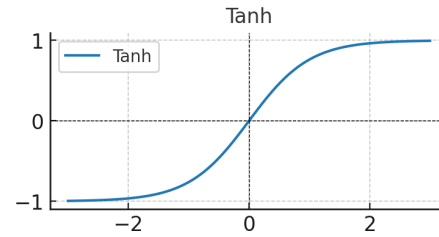
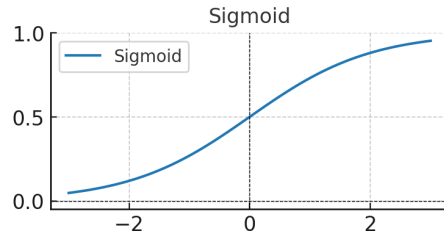
# 1. Evrişim Katmanı
model.add(Conv2D(
    filters=32,           # Filtre sayısı
    kernel_size=(3, 3),   # Filtre boyutu
    activation='relu',     # Aktivasyon fonksiyonu
    input_shape=(28, 28, 1) # Giriş verisi boyutu (28x28, gri tonlamalı)
))
```

Evrişim Katmanı + Aktivasyon Fonksiyonu (ReLU)

Negatif değerleri sıfırlar, pozitif değerleri olduğu gibi bırakır. Genellikle evrişimli sinir ağları (CNN) ve beslemeli ileri ağlar (Feedforward Neural Networks) dahil olmak üzere derin öğrenme modellerinin gizli katmanlarında kullanılır.



Aktivasyon Fonksiyonları



$$f(x) = x \cdot \text{sigmoid}(x)$$

$$f(x) = x \cdot \Phi(x)$$

$$f(x) = x$$

Aktivasyon Fonksiyonları

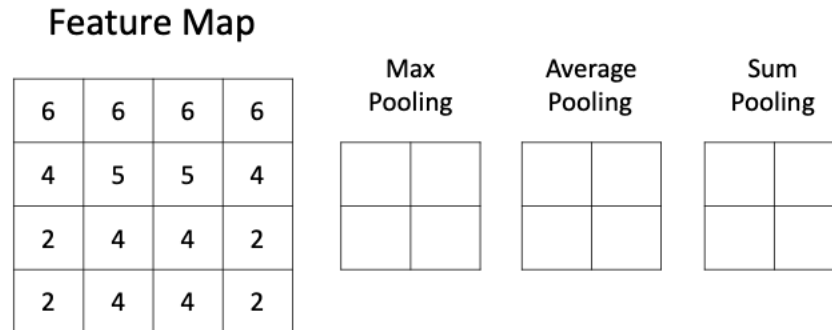
Fonksiyon	Avantajları	Dezavantajları	Kullanım Alanları
Sigmoid	Probabilistik sonuçlar, Basit	Gradyan sönmesi problemi	İkili sınıflandırma (çıkış katmanı)
Tanh	Sıfır merkezli, Güçlü öğrenme	Gradyan sönmesi problemi	RNN ve orta katmanlar
ReLU	Basit, Gradyan sönmesi yok, Hızlı	Ölü nöron sorunu	Gizli katmanlar (CNN, DNN)
Leaky ReLU	Ölü nöron yok, Negatif değerleri işler	Hiperparametre (α) gerektirir	Gizli katmanlar (Ölü nöron sorununda)
ELU	Sıfır merkezli, Negatif değerleri işler	Hesaplama maliyeti yüksektir	Derin sinir ağları
Softmax	Olasılık dağılımı sağlar, Çok sınıflı	Gradyan sönmesi, Yüksek maliyet	Çok sınıflı sınıflandırma
Swish	Gradyan sönmesini azaltır, Düzgün geçiş	Hesaplama maliyeti yüksektir	Derin öğrenme ağları
GELU	Güçlü doğrusal olmayan geçişler sağlar	Hesaplama maliyeti yüksektir	Transformer modelleri (BERT)
Maxout	Esnek, Birden fazla fonksiyonu birleştirir	Daha fazla parametre gerektirir	Özel esneklik gerektiren modeller
Linear	Basit, Doğrusal problemler için ideal	Doğrusal olmayan problemler için uygun değildir	Regresyon problemleri

Havuzlama Katmanı (Pooling Layer)

- Özellik haritalarını (konvolüsyon işlemlerinden sonra üretilen) alt örneklemek için kullanılır, yani daha büyük boyutlu özellik haritalarını alır ve bunları daha küçük boyutlu özellik haritalarına küçültür.
- Özellik haritalarını küçültürken, her havuz adımında en baskın özellikleri (veya bilgileri) daima korur.
- Havuzlama işlemi, konvolüsyon işlemine benzer şekilde havuzlanmış bölge boyutu ve işlemin adımını belirterek gerçekleştirilir.
- Maksimum havuzlama, minimum havuzlama, ortalama havuzlama gibi farklı havuzlama teknikleri vardır. Max Havuzlama en popüler ve en çok kullanılan havuzlama tekniğidir.

Havuzlama Katmanı (Pooling Layer)

- Havuzlama katmanının ana dezavantajı, bazen CNN'in genel performansını düşürmesidir. Bunun nedeni, havuzlama katmanının CNN'in verilen giriş görüntüsünde belirli bir özelliğin bulunup bulunmadığını, bu özelliğin doğru konumunu önemsemeden bulmasına yardımcı olmasıdır.



Step-1

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1

max (1,0,-1,0)

1		

Step-2

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1

max (0,-2,0,1)

1	1	

Step-3

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1

max (-2,1,1,2)

1	1	2

Step-4

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1

max (-1,0,0,2)

1	1	2
2		

Finally

1	0	-2	1
-1	0	1	2
0	2	1	0
1	0	0	1

After performing the
complete Max Pooling
Operation

1	1	2
2	2	2
2	2	1

$$h' = \left\lfloor \frac{h - f}{s} \right\rfloor$$

$$w' = \left\lfloor \frac{w - f}{s} \right\rfloor$$

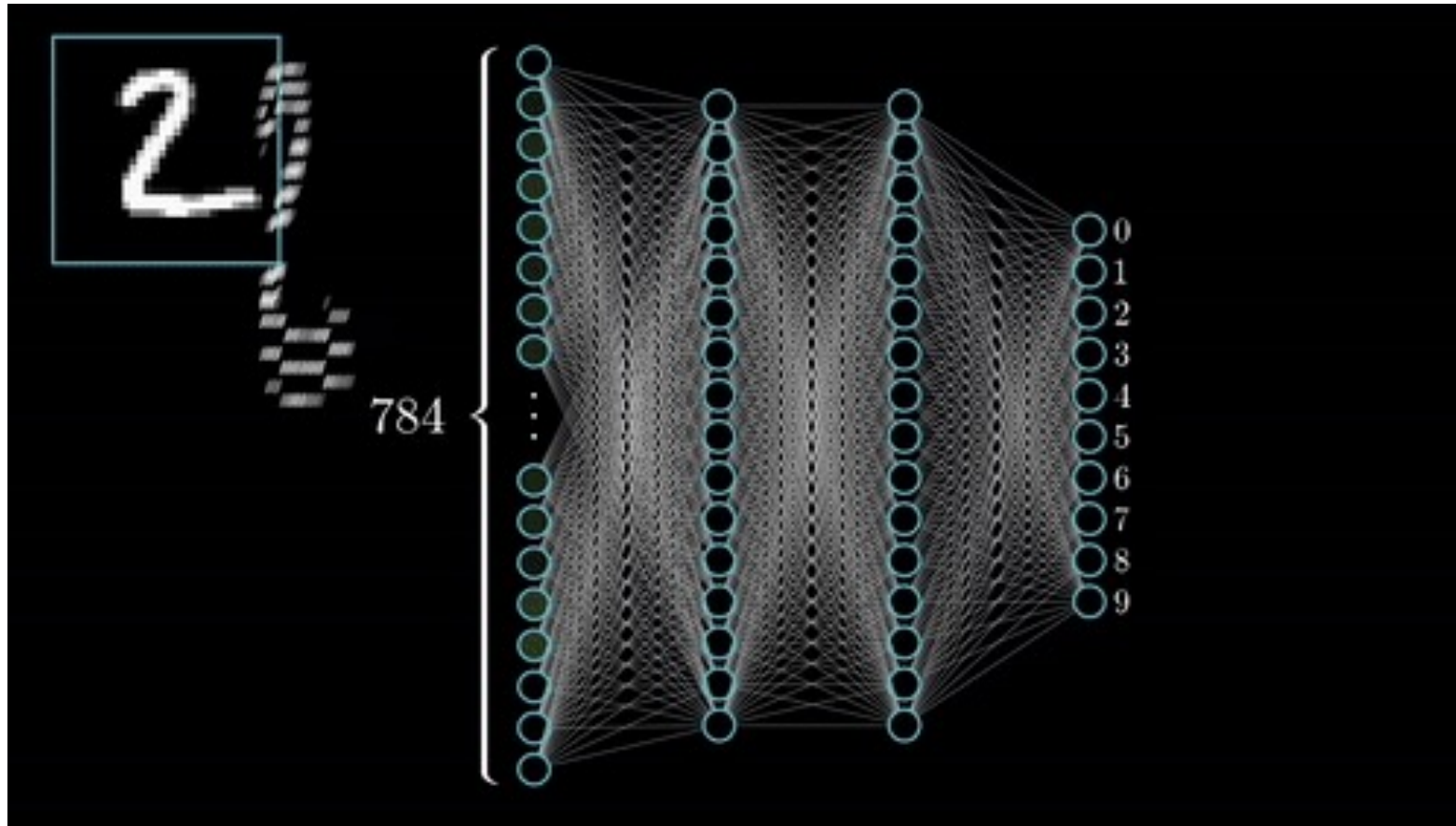
Tam Bağlı Katman (Fully Connected Layer)(FC)

- Genellikle her CNN mimarisinin (sınıflandırma için kullanılan) son kısmı (veya katmanları) şunlardan oluşur ;
- Tam bağlantılı katmanlar, bir katman içindeki her nöronun varsa kendinden önceki ve sonraki katmanındaki her nöronla bağlantılı olduğu katmanlardır.
- Tam Bağlantılı katmanların son katmanı çıktı katmanı (sınıflandırıcı) olarak kullanılır ve CNN mimarisinin bir parçasıdır.

Tam Bağlı Katman (Fully Connected Layer)(FC)

- Tam Bağlantılı Katmanlar, ileri beslemeli yapay sinir ağı (YSA) türüdür ve geleneksel çok katmanlı algılayıcı sinir ağı (MLP) prensibini takip eder.
- FC katmanları, son konvolüsyonel veya havuzlama katmanından girdi alır; bu girdi bir dizi metrik şeklindedir (özellik haritaları) ve bu metrikler bir vektör oluşturmak için düzleştirilir. Bu vektör daha sonra CNN'in nihai çıktısını oluşturmak için FC katmanına beslenir.

Tam Bağlı Katman (Fully Connected Layer)(FC)



CNN-Python

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
model = Sequential([
    Conv2D(64, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),

    Conv2D(128, (3, 3), activation='relu'),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),

    Flatten(),
    Dense(512, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])
```

Kayıp Fonksiyonları

- Çıktı katmanında, CNN modelinin eğitim örnekleri üzerinde ürettiği tahmin hatasını bazı Kayıp Fonksiyonları kullanarak hesaplarız. Bu tahmin hatası, ağa tahminlerinin gerçek çıktıdan ne kadar uzak olduğunu söyler.
- Kayıp fonksiyonu hatayı hesaplamak için iki parametre kullanır, ilk parametre CNN modelinin tahmini çıktısıdır (tahmin olarak da bilinir) ve ikincisi gerçek çıktıdır (etiket olarak da bilinir).

Kayıp Fonksiyonları

Kayıp Fonksiyonu	Kullanım Alanları	Avantajları	Dezavantajları
Mean Squared Error (MSE)	Regresyon problemleri	Basit ve hesaplaması kolay.	Aykırı değerlere karşı duyarlıdır.
Mean Absolute Error (MAE)	Regresyon problemleri	Daha az aykırı değerlere duyarlı.	Türev hesaplaması nedeniyle optimizasyonu daha zordur.
Categorical Crossentropy	Çok sınıflı sınıflandırma	Çok sınıflı problemler için ideal.	Büyük sınıf dengesizliklerinde performansı düşebilir.
Binary Crossentropy	İkili sınıflandırma problemleri	İkili sınıflandırma için uygundur.	Çıkış aktivasyonunun sigmoid olması zorunludur.
Huber Loss	Regresyon problemleri	Aykırı değerlere karşı dayanıklıdır.	Hiperparametre (δ) ayarı gerektirir.
Kullback-Leibler Divergence (KL)	Olasılık dağılımı modelleme	Dağılımlar arasındaki farkı ölçmek için uygundur.	Hesaplaması daha karmaşıktır.
Hinge Loss	Destek vektör makineleri (SVM)	Hatalı sınıflandırmaların cezasını artırır.	Özellikle SVM'ler için tasarlanmıştır.
Sparse Categorical Crossentropy	Çok sınıflı sınıflandırma (etiketler tek sayı)	Çok sınıflı sınıflandırma için uygundur.	Etiketlerin tek sayı olarak verilmesi gerekir.
Poisson Loss	Sayı sayma problemleri	Sayı tahmini ve zaman serisi problemleri için uygundur.	Verilerdeki varyansın sabit olması gerekir.
Cosine Similarity Loss	Vektörlerin benzerlik ölçümü	İkili vektör benzerliği ölçümü için uygundur.	Büyük boyutlu vektörlerde hesaplama maliyeti yüksek olabilir.

CNN'de Eğitim Süreci

- Eğitim süreci temel olarak aşağıdaki adımları içerir:
- Veri ön işleme ve Veri artırma.
- Parametre başlatma.
- CNN'in düzenli hale getirilmesi.
- Optimize edici seçimi.

Veri Ön İşleme ve Veri Arttırma

- Veri kümesini daha temiz, daha özellikli, daha öğrenilebilir ve tek tip formatlı hale getirmek için ham veri kümesinde (eğitim, doğrulama ve test veri kümeleri dahil) bazı yapay dönüşümler gerçekleştirme anlamına gelmektedir.
- Veri ön işleme, verileri CNN modeline beslemeden önce yapılır.
- Evrişimli bir sinir ağında, CNN'in performansının onu eğitmek için kullanılan veri miktarıyla doğru orantılı olduğu bir gerçektir, yani iyi bir ön işleme, modelin doğruluğunu her zaman artırır. Ancak diğer taraftan, kötü bir ön işleme de modelin performansını düşürebilir.

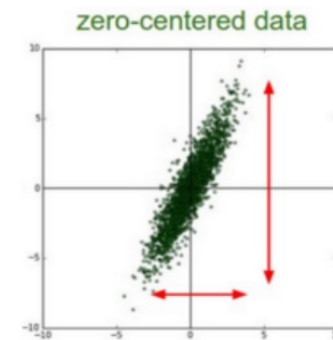
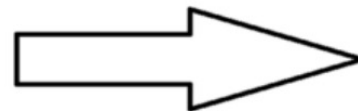
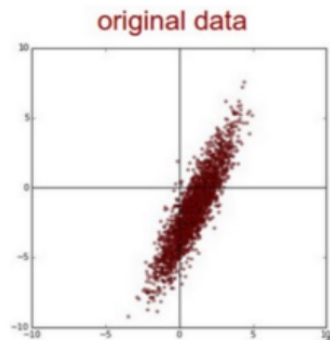
Veri Önışleme ve Veri Arttırma

- **Mean-subtraction (Zero centering)(Ortalama Çıkarma):** Bir veri kümesindeki her bir veriden (örneğin piksel değerlerinden) ortalama değeri çıkararak veriyi sıfır etrafında merkezler.

$$X' = X - x^*$$

$$\text{And, } x^* = \frac{1}{N} \sum_{i=1}^N x_i$$

```
import numpy as np
# Örnek bir veri kümesi (MNIST gibi)
data = np.random.rand(1000, 28, 28, 1) # 1000 adet 28x28 gri görüntü
mean_pixel_value = np.mean(data, axis=(0, 1, 2)) # Ortalama piksel değeri
data_zero_centered = data - mean_pixel_value
print(f"Original Ortalama: {np.mean(data)}")
print(f"Zero-Centered Ortalama: {np.mean(data_zero_centered)}")
```

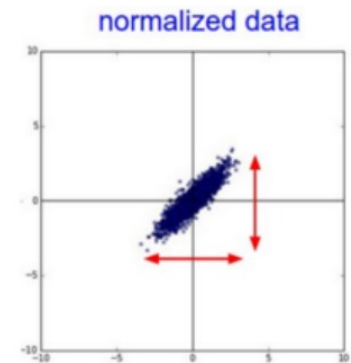
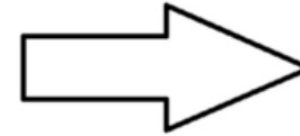
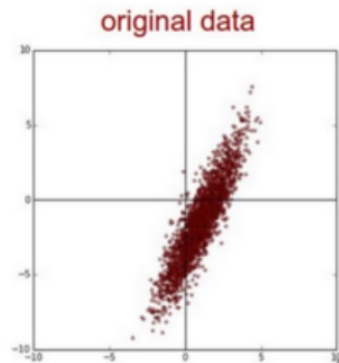


Veri Önışleme ve Veri Arttırma

- **Normalizasyon:** Giriş verisini belirli bir aralığa ([0 , 1] [0,1] veya [- 1 , 1] [-1,1]) getirerek modelin daha hızlı öğrenmesini sağlar.

$$X'' = \frac{X'}{\sqrt{\frac{\sum_{i=1}^N (x_i - x^*)^2}{N-1}}}$$

```
import numpy as np
data = np.random.randint(0, 256, size=(1000, 28, 28)) # 1000 görüntü, 28x28 boyutunda
normalized_data = data / 255.0
print(f"Original Veri Min: {data.min()}, Max: {data.max()}")
print(f"Normalize Edilmiş Veri Min: {normalized_data.min()}, Max: {normalized_data.max()}")
```



Veri Önışleme ve Veri Arttırma

- **Standardizasyon:** Girdilerin ortalamasını 0, standart sapmasını 1 yaparak veriyi ölçeklendirir.

$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

```
mean = np.mean(data, axis=(0, 1, 2))
std = np.std(data, axis=(0, 1, 2))
standardized_data = (data - mean) / std
print(f"Orijinal Veri Ortalama: {mean:.2f}, Standart Sapma: {std:.2f}")
print(f"Standardize Edilmiş Veri Ortalama: {np.mean(standardized_data):.2f}, Standart Sapma: {np.std(standardized_data):.2f}")
```

Veri Ön işleme ve Veri Arttırma

- Veri Arttırma (Data Augmentation): Eğitim verisini yapay olarak genişleterek modelin genelleme kapasitesini artırmak. Türleri aşağıdaki gibidir:
- **Döndürme (Rotation):** Görüntüleri belirli bir açıyla döndürme.
- **Çevirme (Flipping):** Görüntüyü yatay veya dikey ekseninde çevirmek.
- **Kesme (Cropping):** Görüntünün bir bölümünü kırparak eğitim yapmak.
- **Gürültü Ekleme (Adding Noise):** Rastgele gürültü eklemek.
- **Parlaklık ve Kontrast Değişimi:** Görüntünün parlaklığını veya kontrastını değiştirmek.

Veri Arttırma (Data Augmentation)- Generator Kavramı

- Generator yani "üreteç" olarak anılmasının nedeni, verileri ihtiyaç duyuldukça üreterek bellekte gereksiz bir yük oluşturmaması ve büyük veri kümeleriyle verimli çalışmasıdır.
- Python'da "generator" kelimesi, bir dizi veriyi adım adım üretme yeteneğine sahip bir yapı anlamına gelir.
- Hafıza dostu bir yaklaşımdır çünkü veri kümesini bir defada belleğe yüklemek yerine, ihtiyaca göre parça parça veriyi üretir.

Train Generator (Eğitim Üreteci)

- Eğitim verilerini küçük partiler (batch'ler) halinde sunar ve model eğitimi sırasında kullanılır.
- Büyük veri setlerini bir defada belleğe yüklemek yerine, bu generator veri akışını sağlar ve veri artırımı (augmentation) gibi teknikleri uygulayabilir.
- Bir modelin eğitimi sırasında, ImageDataGenerator gibi sınıflar aracılığıyla eğitim verileri için bir üreteç oluşturabilir.

Validation Generator(Doğrulama Üreteci)

- Model eğitimi sırasında doğrulama yapmak için kullanılır.
- Modelin aşırı uyum (overfitting) veya diğer sorunlara karşı doğrulanmasını sağlar. Ayrıca, modelin genel performansını değerlendirir.
- Modelin eğitimi sırasında, doğrulama verileri için kullanılır.

Test Generator(Test Üreteci)

- Modelin eğitimi tamamlandıktan sonra, test verilerini model üzerinde test etmek için kullanılır.
- Modelin gerçek dünya performansını değerlendirmek için test verilerine ihtiyaç vardır. Bu generator, test verilerini bellek verimliliği ile sağlayarak büyük veri setleriyle çalışmayı kolaylaştırır.
- Model eğitimi tamamlandıktan sonra, modelin test edilmesi için kullanılır.

Data Generator(Veri Üreteci)

- Genel olarak veri akışını sağlamak için kullanılır. Örneğin, özel veri tabanlarından veya dosya sistemlerinden veri akışı sağlamak için kullanılabilir.
- Verinin farklı kaynaklardan gelmesini sağlar ve büyük veri setleriyle çalışırken bellek kullanımını optimize eder.
- CSV dosyaları veya özel veri kümeleri gibi farklı kaynaklardan veri akışını sağlamak için kullanılır.

Custom Generator(Özel Üreteç)

- Özelleştirilmiş veri akışları oluşturmak için kullanılır. Örneğin, veri setinin belirli bir kısmını çekmek, özel veri işleme adımları uygulamak, veya çoklu veri kaynaklarını birleştirmek için kullanılabilir.
- Belirli gereksinimlere göre veri akışı sağlamak için esnek bir çözüm sunar.
- Özel veri işleme adımları veya veri artırımı teknikleri uygulamak için kullanılır.

ImageDataGenerator

Özellik/Fonksiyon	Açıklama
rescale	Tüm görüntülerin değerlerini ölçeklendirmek için kullanılır. Genellikle 1/255 olarak ayarlanır, böylece pikseller 0-255 aralığından 0-1 aralığına dönüşür.
featurewise_center	Tüm veri kümesi genelinde ortalama çıkarılarak görüntüleri merkezler.
featurewise_std_normalization	Tüm veri kümesi genelinde standart sapmaya göre görüntüleri normalleştirir.
zca_whitening	Görüntüleri ZCA (Zero-phase Component Analysis) ile beyazlatır.
rotation_range	Görüntüleri rastgele bir açı aralığında döndürür.
width_shift_range	Görüntüleri yatayda rastgele kaydırır (oransal).
height_shift_range	Görüntüleri dikeyde rastgele kaydırır (oransal).
shear_range	Görüntülere rastgele makaslama (shear) uygular.
zoom_range	Görüntüleri rastgele yakınlaştırır.
channel_shift_range	Renk kanallarında rastgele kaydırma yapar.
horizontal_flip	Görüntüleri yatay olarak rastgele çevirir.
vertical_flip	Görüntüleri dikey olarak rastgele çevirir.
fill_mode	Görüntüleri kaydırırken veya döndürürken kenarları doldurma şekli ('nearest', 'constant', 'reflect', 'wrap').
cval	fill_mode 'constant' olduğunda, doldurma için kullanılacak değer.
preprocessing_function	Her bir görüntü üzerinde çalıştırılacak özel bir işlev.
data_format	Görüntü veri formatını belirler ('channels_first', 'channels_last').
validation_split	Eğitim ve doğrulama verilerini ayırmak için oran.

Parametre Başlatma

- Derin bir CNN milyonlarca veya milyarlarca parametreden oluşur. Bu nedenle, eğitim sürecinin başında iyi bir şekilde başlatılması gerekir, çünkü ağırlık başlatma, CNN modelinin ne kadar hızlı ve ne kadar doğru sonuç verebileceğini doğrudan belirler.

Rastgele Başlatma

- Ağırlıkları rastgele matrisler kullanarak rastgele başlatır. Ancak rastgele başlatmanın temel sorunu, potansiyel olarak kaybolan gradyanlar veya patlayan gradyanlar sorunlarına yol açabilmesidir. Bazı popüler rastgele başlatma yöntemleri şunlardır:
- **Gauss Rastgele Başlatma:** Burada ağırlıklar rastgele bir matris kullanılarak rastgele başlatılır ve bu matrisin elemanları bir Gauss dağılımından örneklenir.
- **Tekdüze Rastgele Başlatma:** Burada ağırlıklar rastgele bir Tekdüze matris kullanılarak rastgele başlatılır ve bu matrisin elemanları tekdüze bir dağılımdan örneklenir.
- **Ortogonal Rastgele Başlatma:** Burada ağırlıklar rastgele ortogonal matrisler kullanılarak rastgele başlatılır ve bu matrislerin elemanları ortogonal bir dağılımdan örneklenir.

Xavier Başlatma

- Bu teknik Xavier Glorot ve Yoshua Bengio tarafından 2010 yılında önerilmiştir, ağdaki her katman için çıkış bağlantılarının ve giriş bağlantılarının varyansını eşit hale getirmeye çalışır. Ana fikir, aktivasyon fonksiyonlarının varyansını dengelemektir.

Denetimsiz Ön-Eğitim Başlatma

- Bir evrişimli sinir ağını başka bir evrişimli sinir ağıyla (denetimsiz bir teknik kullanılarak eğitilmiş), örneğin derin bir oto kodlayıcı veya derin bir inanç ağıyla başlatırız. Bu yöntem bazen hem optimizasyon hem de aşırı uyum sorunlarının üstesinden gelmeye yardımcı olarak çok iyi çalışabilir.

Kaybolan Gradyan Problemi (Vanishing Gradient)

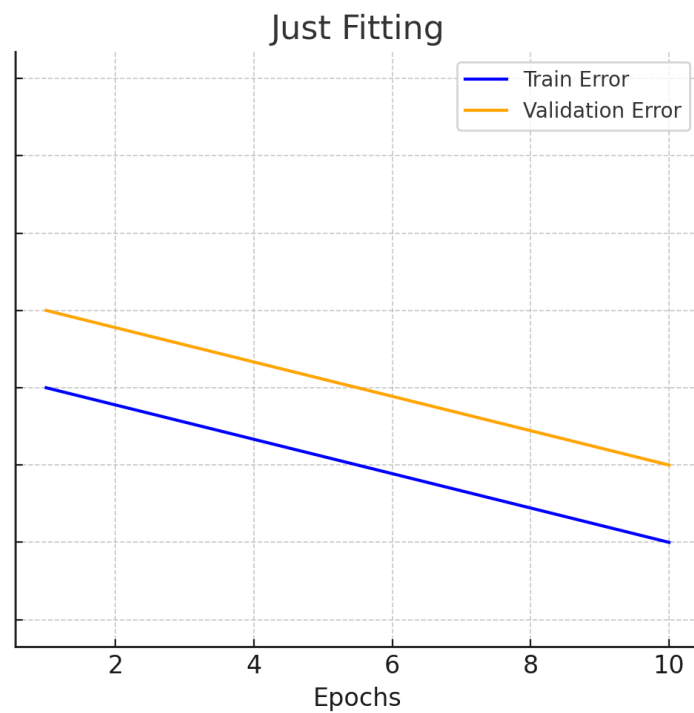
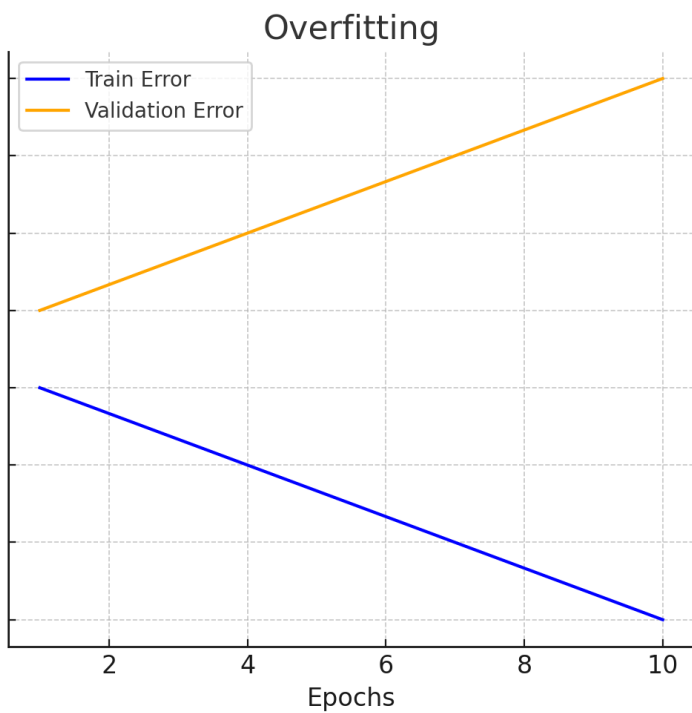
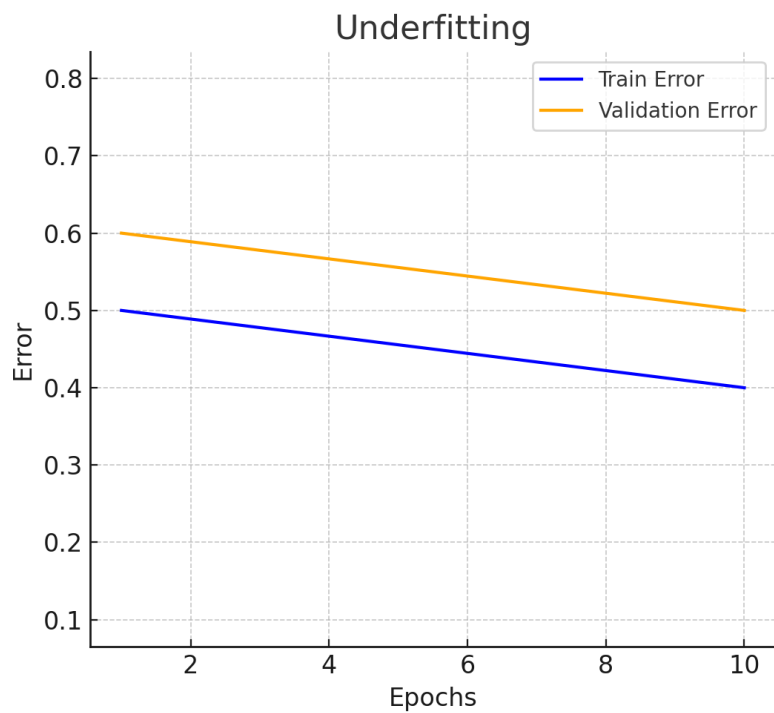
- Derin bir evrişimli sinir ağı (örneğin, 1000 katmanlı bir CNN) üzerinde çalıştığımızda, ağırlıkları güncellemek için her katmandaki nöronların ağırlıklarına göre kayıp fonksiyonunun gradyanlarını (türevlerini) hesaplamamız gerekir. Bu işlem, geri yayılım (backpropagation) sırasında gerçekleşir ve gradyanlar, her bir katman boyunca geriye doğru hesaplanır.
- Ancak, ağda geriye doğru ilerledikçe, gradyanların değerleri küçülmeye başlar. Hatta bazı durumlarda gradyanlar neredeyse sıfıra kadar düşebilir.

Patlayan Gradyan Problemi (Exploding Gradient)

- Kaybolan gradyan probleminin tam tersidir, burada büyük hata gradyanları geri yayılım sırasında birikir ve ağırlıklarında çok büyük güncellemelerle sonuçlanır.
- Modeli kararsız hale getirir ve daha sonra bu kararsız model verimli bir şekilde öğrenemez.

CNN Düzenleme (Regularization)

- Derin öğrenme algoritmalarının temel zorluğu, eğitim verileriyle aynı dağılımdan alınan yeni veya daha önce görülmemiş girdilere düzgün bir şekilde uyum sağlamaktır, bunu yapabilme yeteneğine genelleme denir. Bir CNN modelinin iyi bir genelleme elde etmesindeki ana sorun aşırı uyumdur.



Dropout

- Bırakma anlamındadır. Belirli bir olasılıkla nöronları rastgele devre dışı bırakır. Modelin bağımsız özellikler öğrenmesini teşvik eder.

```
from tensorflow.keras.layers import Dropout  
model.add(Dropout(0.5))
```

L2 Regularization

- Ağırlıkların büyüklüğüne bir ceza ekler. Ağırlıkların küçük değerler almasını sağlar.

```
from tensorflow.keras.regularizers import l2
model.add(Dense(128, activation='relu', kernel_regularizer=l2(0.01)))
```


L1 Regularization

- Ağırlıkların toplamına bir ceza ekler. Gereksiz ağırlıkları tamamen sıfıra indirir (özellik seçimi sağlar).

```
from tensorflow.keras.regularizers import l1  
model.add(Dense(128, activation='relu', kernel_regularizer=l1(0.01)))
```

Early Stopping

- Eğitim sırasında doğrulama kaybı iyileşmediğinde eğitimi durdurur. Gereksiz epoch'larda aşırı öğrenmeyi engeller.

```
from tensorflow.keras.callbacks import EarlyStopping
early_stopping = EarlyStopping(monitor='val_loss', patience=5)
model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=100, callbacks=[early_stopping])
```

Batch Normalization

- Eğitim sırasında doğrulama kaybı iyileşmediğinde eğitimi durdurur. Gereksiz epoch'larda aşırı öğrenmeyi engeller.

```
from tensorflow.keras.callbacks import EarlyStopping
early_stopping = EarlyStopping(monitor='val_loss', patience=5)
model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=100, callbacks=[early_stopping])
```

Optimizer Seçimi

- **RMSProp**: Adagrad'ın öğrenme hızının sürekli düşme problemini çözer. RMSProp, yalnızca bir dizi son gradyanın karesini hesaba katar, bu nedenle daha yeni gradyanlara daha fazla ağırlık verir.
- **Adam (Adaptive Moment Estimation)**: Hem Momentum hem de RMSProp'un avantajlarını birleştirir. Adam, hem ilk anları (Momentum gibi) hem de ikinci anları (RMSProp gibi) hesaba katar.
- **Adadelta** ve **Adamax** gibi diğer bazı optimizasyon yöntemleri de mevcuttur. Bunlar genellikle belirli durumlar için daha uygun olabilirler.

Optimizasyon Algoritmaları (Optimizer'lar)

Optimizasyon Algoritması	Açıklama	Kullanım Örneği
Adam	Öğrenme oranını adaptif olarak ayarlar ve momentum içerir	optimizer='adam'
SGD	Standart stokastik gradyan inişi, momentum seçeneğiyle	optimizer=keras.optimizers.SGD(momentum=0.9)
RMSprop	Gradyanların karelerinin hareketli ortalamasını kullanır	optimizer='rmsprop'
Adagrad	Öğrenme oranını adaptif olarak ayarlar	optimizer='adagrad'
Adadelat	Adagrad'a benzer ancak öğrenme oranını farklı şekilde ayarlar	optimizer='adadelat'
Adamax	Adam'ın bir uzantısı, infinity normunu kullanır	optimizer='adamax'
Nadam	Adam ve Nesterov momentumunu birleştirir	optimizer='nadam'
Ftrl	FTRL (Follow The Regularized Leader), büyük ölçekli linear modeller için kullanılır	optimizer='ftrl'

Eğitim, Doğrulama, Test Verileri

1.Eğitim Verisi:

- Modelin eğitiminde kullanılan veri kümesidir.
- Modelin ağırlıklarını öğrenmesi ve eğitilmesi için kullanılır.

2.Doğrulama Verisi:

- Eğitim sırasında, modelin performansını ara sıra kontrol etmek için kullanılır.
- Hiperparametre optimizasyonu ve erken durdurma gibi stratejiler için kullanışlıdır.

3.Test Verisi:

- Modelin eğitimi tamamlandıktan sonra gerçek performansını değerlendirmek için kullanılır.
- Modelin genelleme yeteneğini ölçer ve eğitim sırasında modelin aşırı öğrenme (overfitting) yapıp yapmadığını gösterir.

Eğitim ve Doğrulama Veri Seti Özellikleri

Özellik	Açıklama
target_size	Görüntülerin yeniden boyutlandırıldığı hedef boyut. Örneğin, (128, 128) şeklinde boyut verilir.
batch_size	Her eğitim adımında işlenen veri sayısı. Çoğunlukla 32 veya 64 gibi değerler kullanılır.
class_mode	Etiket türü. 'categorical', 'binary', 'sparse', veya 'input'.
subset	Veri kümesinin eğitim veya doğrulama olarak ayrılmasını belirtir. 'training' veya 'validation'.
shuffle	Veri setinin karıştırılması. True veya False. Eğitimde karıştırmak genelde daha iyidir.
rescale	Görüntü verilerini normalleştirmek için kullanılan ölçek faktörü. Örneğin, 1.0/255.
rotation_range	Görüntüleri rastgele döndürmek için kullanılan derece aralığı. Örneğin, 0-180 derece.
width_shift_range	Görüntüleri yatay olarak kaydırma miktarı. Örneğin, 0.1 = %10 kaydırma.
height_shift_range	Görüntüleri dikey olarak kaydırma miktarı.
zoom_range	Görüntülerin rastgele yakınlaştırılması için kullanılan faktör.
horizontal_flip	Görüntüleri yatay olarak rastgele çevirme. True veya False.
validation_split	Eğitim ve doğrulama verilerini ayırmak için kullanılan oran. Örneğin, 0.2 = %20 doğrulama.

Katmanlar Arası Veri Boyutu

Katman Türü	Parametreler	Hesaplama Formülü	Örnek Hesaplama
Evrişim (Convolution)	- Giriş Boyutu: (128, 128, 3) - Filtre Boyutu (Kernel): (3, 3) - Doldurma (Padding): 'same' - Adım (Stride): 1	- Çıkış Yüksekliği: $\text{output height} = \frac{\text{input height} - \text{filter height} + 2 * \text{Padding}}{\text{stride}} + 1$ - Çıkış Genişliği: $\text{output width} = \frac{\text{input width} - \text{filter width} + 2 * \text{Padding}}{\text{stride}} + 1$	- Çıkış Boyutu: Yükseklik ve genişlik değişmez, çünkü 'same' padding kullanılır. Çıkış boyutu (128, 128, 32) olur.
Havuzlama (Pooling)	- Giriş Boyutu: (128, 128, 32) - Havuzlama Boyutu (Pooling Size): (2, 2) - Adım (Stride): 2	- Çıkış Yüksekliği ve Genişliği: $\text{output height} = \frac{\text{input height}}{\text{stride}}$ $\text{output width} = \frac{\text{input width}}{\text{stride}}$	- Çıkış Boyutu: Yükseklik ve genişlik havuzlama nedeniyle yarıya iner. Çıkış boyutu (64, 64, 32) olur.

Katmanlar

Katman İsmi	Ne İşe Yarar	Örnek Kullanım Kodu
Conv2D	2D görüntüler üzerinde evrişim (convolution) uygulamak için kullanılır. Görüntüden özellikleri çıkarmak için filtrelerle çalışır.	<code>layers.Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3))</code>
MaxPooling2D	Birlikte olan iki boyutlu havuzlama uygular. Özellikle özellikleri indirgeyerek hesaplama maliyetini düşürmek ve fazla öğrenmeyi önlemek için kullanılır.	<code>layers.MaxPooling2D((2, 2), strides=2)</code>
Flatten	Çok boyutlu bir tensörü düz bir diziye dönüştürür. Genellikle evrişim katmanları sonrası kullanılacak yoğun (dense) katmanlara geçiş yapmak için kullanılır.	<code>layers.Flatten()</code>
Dense	Geleneksel tam bağlı (fully connected) katmandır. Bir girdi vektörünü başka bir vektöre eşler. Genellikle ağırlı son katmanında sınıflandırma yapmak için kullanılır.	<code>layers.Dense(128, activation='relu')</code>
Dropout	Girdi nöronlarını rastgele belirli bir oranda kapatır (örneğin, %50). Fazla öğrenmeyi önlemek için kullanılır.	<code>layers.Dropout(0.5)</code>
BatchNormalization	Katmanlardaki girdi verisini normalleştirir. Eğitim sürecinde istikrar sağlar ve öğrenme hızını artırır.	<code>layers.BatchNormalization()</code>
Activation	Nöronların çıktısına aktivasyon fonksiyonu uygular. Aktivasyon, katmanın doğrusal olmayan bir yanıt vermesini sağlar.	<code>layers.Activation('relu')</code>
Conv2DTranspose	2D evrişimlerin tersi gibi çalışır. Özellikle yükseltmek (upsample) için kullanılır. Genellikle görüntü sentezi veya görüntü çözünürlüğünü artırmak için kullanılır.	<code>layers.Conv2DTranspose(64, (3, 3), strides=2, padding='same')</code>

Ölçümler

- Accuracy (Doğruluk Oranı):** Modelin doğru tahminlerinin oranıdır. 0 ile 1 arasında değişir, 1 en yüksek doğruluktur. % olarak da ifade edilebilir.
- Loss (Kayıp):** Modelin tahminlerinin gerçeğe olan uzaklığını ölçer. Düşük değerler daha iyidir.
- Validation Accuracy (Doğrulama Doğruluğu):** Modelin doğrulama veri setinde ne kadar doğru tahmin yaptığını gösterir. Eğitim sürecinde modelin genel performansını değerlendirmek için kullanılır.
- Validation Loss (Doğrulama Kaybı):** Modelin doğrulama veri setindeki tahminlerinin yanlışlık derecesini gösterir.

Ölçüm	Anlamı	Açıklama
Epoch	Bir eğitim döngüsü	Modelin eğitim veri setini bir kez baştan sona işlemesi
Accuracy	Doğruluk oranı	Modelin doğru tahmin oranı
Loss	Kayıp	Modelin tahminlerinin ne kadar yanlış olduğunu gösterir
Validation Accuracy	Doğrulama doğruluğu	Modelin doğrulama setindeki doğruluk oranı
Validation Loss	Doğrulama kaybı	Modelin doğrulama setindeki kayıp değeri

Ölçümler (Metrics)

Ölçüm	Açıklama	Kullanım Örneği
Accuracy	Doğruluk oranını hesaplar	metrics=['accuracy']
Precision	Pozitif tahminlerin ne kadarının doğru olduğunu ölçer	metrics=['precision']
Recall	Gerçek pozitiflerden ne kadarının doğru tahmin edildiğini ölçer	metrics=['recall']
F1-Score	Precision ve recall'un harmonik ortalamasını hesaplar	metrics=['f1_score']
AUC	ROC eğrisi altında kalan alanı hesaplar (binary sınıflandırma için)	metrics=['auc']
Mean Squared Error	Ortalama kare hatasını hesaplar (regresyon için)	metrics=['mean_squared_error']
Mean Absolute Error	Ortalama mutlak hatayı hesaplar (regresyon için)	metrics=['mean_absolute_error']
Hinge Loss	Hinge kaybını hesaplar (SVM sınıflandırması için)	metrics=['hinge']

Convolutional Neural Networks (CNNs)- Evrişimli Sinir Ağları

- Birden çok katmandan oluşur ve esas olarak görüntü işleme ve nesne algılama için kullanılır.
- CNN'ler, uydu görüntülerini tanımlamak, tıbbi görüntüleri işlemek, zaman serilerini tahmin etmek ve anormallikleri tespit etmek için yaygın olarak kullanılmaktadır.
- CNN, evrişim işlemini gerçekleştirmek için birkaç filtreye sahip bir evrişim katmanına sahiptir.
- CNN'ler, öğeler üzerinde işlem yapmak için bir ReLU katmanına sahiptir. Çıktı, düzeltilmiş bir özellik haritasıdır.

- Düzeltilmiş özellik haritası daha sonra bir havuzlama katmanına beslenir. Havuzlama, özellik haritasının boyutlarını küçülten bir aşağı örnekleme işlemidir.
- Havuzlama katmanı daha sonra havuzlamış özellik haritasından elde edilen iki boyutlu dizileri düzleştirerek tek, uzun, sürekli, doğrusal bir vektöre dönüştürür.
- Havuzlama katmanından gelen düzleştirilmiş matris, görüntüleri sınıflandıran ve tanımlayan bir girdi olarak beslendiğinde tam bağlantılı bir katman oluşur.